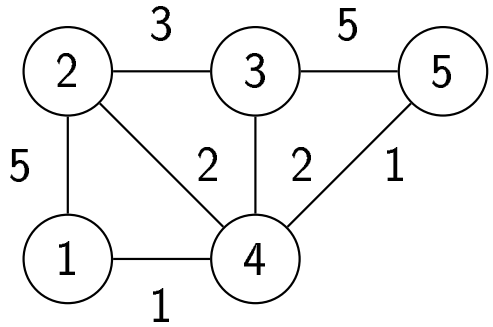
2.2. Minimum cost spanning trees

Optimal spanning trees have a number of applications:

- network design (communication, electrical, ...)
- IP network protocols
- compact memory storage (DNA)
- ...

Example
Design a communication network so as to connect n cities at minimum total cost.

Model
Graph $G = (N, E)$ with $n = |N|$, $m = |E|$, and a cost $c_e \in \mathbb{R}$ for each $e = \{u, v\} \in E$.



Required properties:

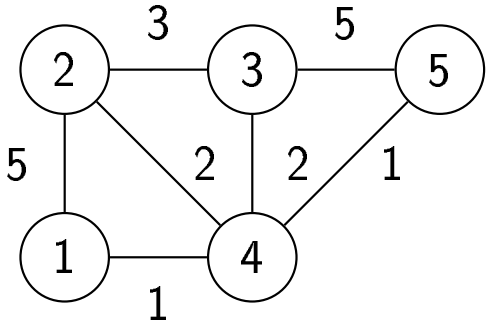
1. Each pair of cities must communicate ⇒ connected subgraph containing all the nodes.
2. Minimum total cost ⇒ subgraph with no cycles.

Problem

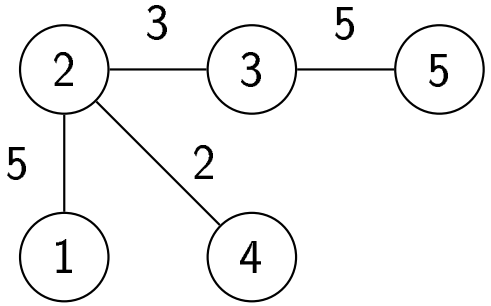
Given an undirected $G = (N, E)$ and a cost $c_e \in \mathbb{R}$ for each $e = \{u, v\} \in E$, find a spanning tree $G_T = (N, T)$ of minimum total cost.

$$\min_{T \in X} \sum_{e \in T} c_e, \quad \text{where } X \text{ is the set of all spanning trees of } G.$$

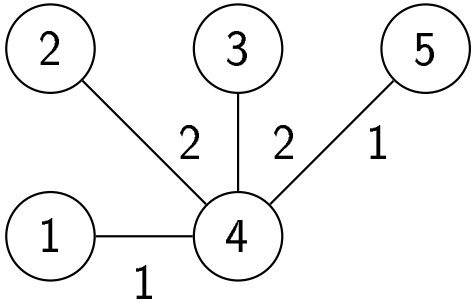
Example



G



$c(T_1) = 15$
(feasible solution)



$c(T_2) = 6$
(feasible and optimal solution)

Theorem 1 (Cayley, 1889)

A complete graph with n nodes ($n \geq 1$) has n^{n-2} spanning trees.

Example

- K_3 ($n = m = 3$) has $3^{3-2} = 3$ spanning trees.
- K_5 ($n = 5$, $m = 10$) has $5^{5-2} = 125$ spanning trees.

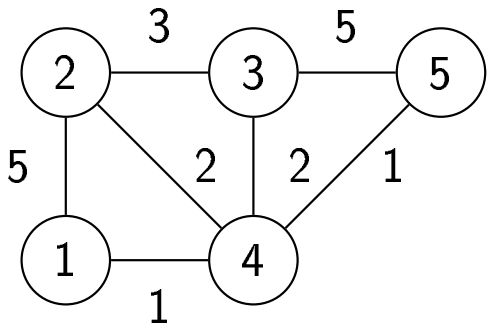
2.2.1 Prim's algorithm

Idea: Iteratively build a spanning tree.

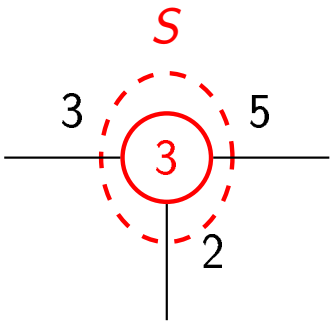
Method

- Start from initial tree (S, T) with $S = \{u\}$ and $T = \emptyset$ (u can be any node in N).
- At each step, add to the current partial tree (S, T) an edge of minimum cost among those which connect a node in S to a node in $N \backslash S$.

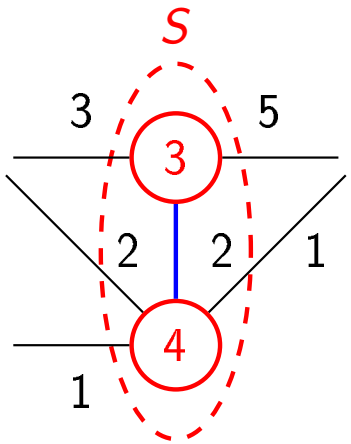
Example



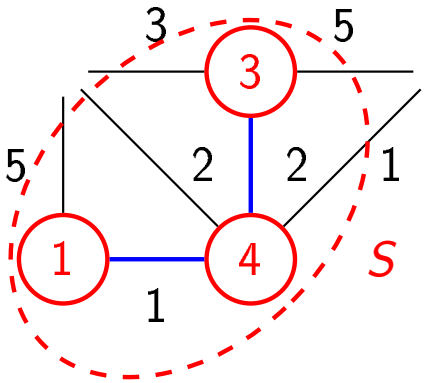
Example



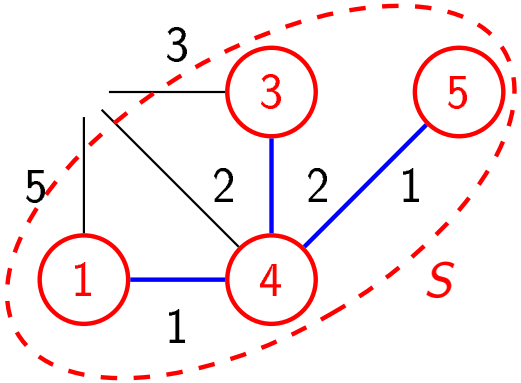
$S = \{3\}$
$T = \emptyset$



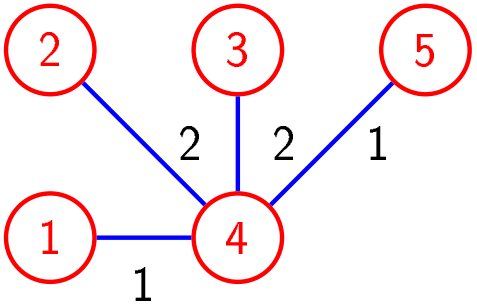
$S = \{3, 4\}$
$T = \{\{3, 4\}\}$



$S = \{1, 3, 4\}$
$T = \{\{3, 4\}, \{1, 4\}\}$



$S = \{1, 3, 4, 5\}$
$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}\}$



$S = N$
$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}, \{2, 4\}\}$
$c(T) = 6$

Prim's algorithm

- Input: Connected $G = (N, E)$ with edge costs
- Output: Subset $T \subseteq N$ of edges of G such that $G_T = (N, T)$ is a minimum cost spanning tree of G.

Algorithm 1: Prim's algorithm for the minimum cost spanning tree problem

```
S ← {u}
T ← ∅
while |T| < n − 1 do
    {u, v} ← edge in δ(S) of minimum cost /* u ∈ S and v ∈ N\S */
    S ← S ∪ {v}
    T ← T ∪ {{u, v}}
```

If all edges are scanned at each iteration, the complexity order is: $O(nm)$.

Exactness of Prim's algorithm

Definition: An algorithm is exact if it provides an optimal solution for every instance, otherwise it is a heuristic.

Proposition 1

Prim's algorithm is exact.

Exact regardless of the choices of the first node and of the edge of minimum cost in $\delta(S)$.

We show that each selected edge belongs to a minimum cost spanning tree.

Property 1 (Cut property)

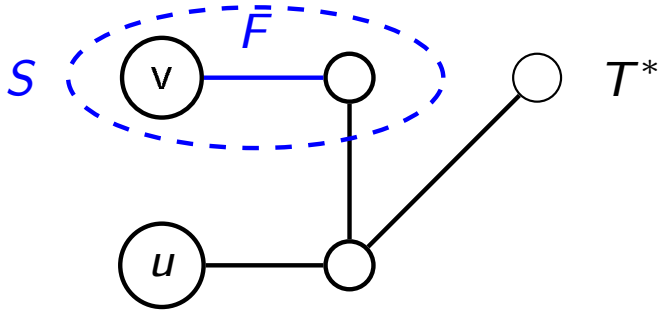
Let F be a partial tree (spanning nodes in $S \subseteq N$) contained in an optimal tree of G. Consider $e = \{u, v\} \in \delta(S)$ of minimun cost, then there exists a minimum cost spanning tree of G containing e.

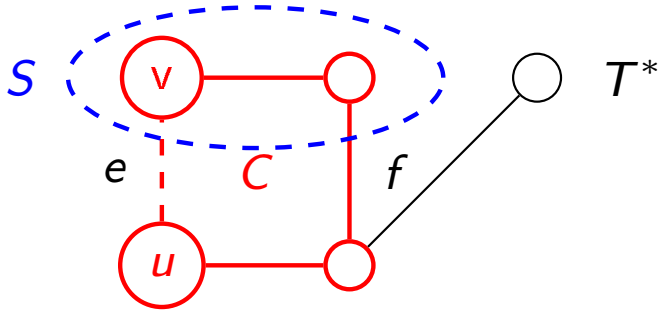
Proof.

By contradiction, assume $T^* \subseteq E$ is a minimum cost spanning tree with $F \subseteq T^*$ and $e \notin T^*$.
Adding edge e to T^* creates a unique cycle C. Let $f \in \delta(S) \cap C$.

- If $c_e = c_f$, then $T^* \cup \{e\} \backslash \{f\}$ is (also) optimal since it has same cost of T^*.
- If $c_e < c_f$, then $c(T^* \cup \{e\} \backslash \{f\}) < c(T^*)$, hence T^* is not optimal.

□





Definition: A greedy algorithm constructs a feasible solution iteratively by making at each step a "locally optimal" choice, without reconsidering previous choices.

Note: Prim's algorithm is greedy.

At each step a minimum cost edge is selected in the cut $\delta(S)$ induced by the current set of nodes S.

Note: For most discrete optimization problems, greedy-type algorithms yield a sub-optimal/poor solution.

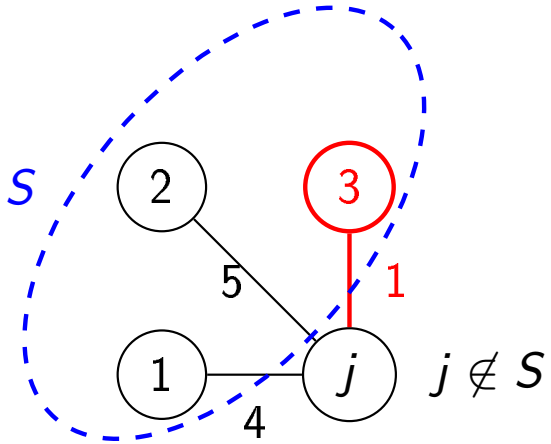
Various greedy algorithms for the minimum cost spanning tree problem:

- Prim
- Kruskal – Exercise 2.2
- ...

Implementation in $O(n^2)$

Data structure

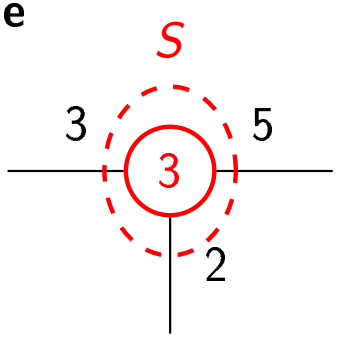
- k: number of edges selected so far
- Subset $S \subseteq N$ of nodes incident to the selected edges
- Subset $T \subseteq E$ of selected edges
- $C_j = \begin{cases} \min\{c_{ij} : \ i \in S\}, & j \notin S \\ +\infty, & \text{otherwise} \end{cases}$
- $closest_j = \begin{cases} \text{argmin}\{c_{ij} : \ i \in S\}, & j \notin S \\ \text{“predecessor” of } j \text{ in the minimum spanning tree,} & j \in S \end{cases}$



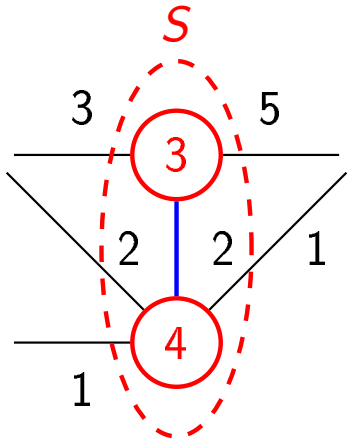
$closest_j = 3$

$C_j = c_{closest_j,j} = 1$

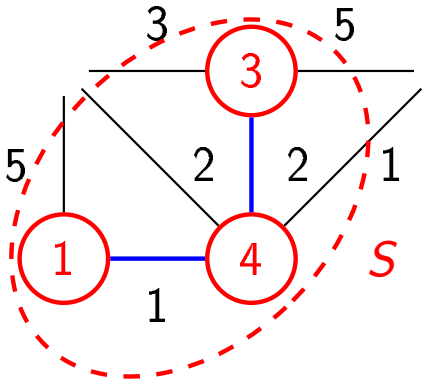
Example



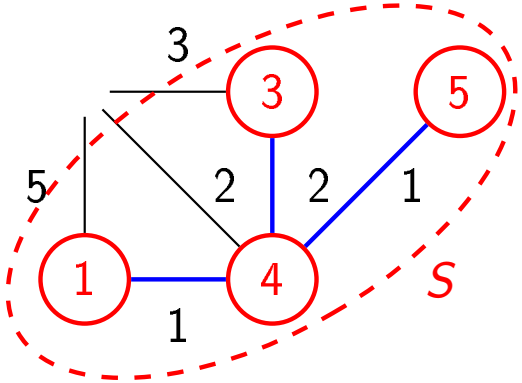
$S = \{3\}$
$T = \emptyset$
$C = [+\infty, 3, -, 2, 5]$
$closest = [3, 3, -, 3, 3]$



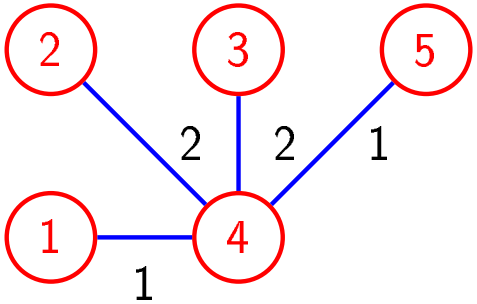
$S = \{3, 4\}$
$T = \{\{3, 4\}\}$
$C = [1, 2, -, 2, 1]$
$closest = [4, 4, -, 3, 4]$



$S = \{1, 3, 4\}$
$T = \{\{3, 4\}, \{1, 4\}\}$
$C = [1, 2, -, 2, 1]$
$closest = [4, 4, -, 3, 4]$



$S = \{1, 3, 4, 5\}$
$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}\}$
$C = [1, 2, -, 2, 1]$
$closest = [4, 4, -, 3, 4]$



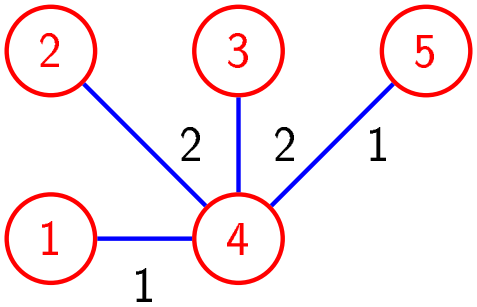
$S = N$
$T = \{\{3, 4\}, \{1, 4\}, \{4, 5\}, \{2, 4\}\}$
$C = [1, 2, -, 2, 1]$
$closest = [4, 4, -, 3, 4]$

How to retrieve the spanning tree from *closest*?

The minimum spanning tree found by Prim's algorithm consists of the $n-1$ edges: $\{closest_j, j\}$, with $j = 1, 2, \ldots, n$ and $j \neq u$.

Example:

Since $closest = [4, 4, -, 3, 4]$, a minimum cost spanning tree consists of the edges: $\{4, 1\}, \{4, 2\}, \{3, 4\}, \{4, 5\}$



$O(n^2)$ version of Prim's algorithm

Algorithm 2: Prim's algorithm

```
T ← ∅; S ← {u}                    /* Initialization                                  */
for j ∈ N\S do
    C_j ← c_uj                    /* If {u,j} ∉ E then c_uj = +∞                     */
    closest_j ← u
for k = 1, ..., n − 1 do
    min ← +∞                      /* Select min edge in δ(S)                         */
    for j = 1, ..., n do
        if j ∉ S and C_j < min then
            min ← C_j; v ← j
    S ← S ∪ {v}; T ← T ∪ {{closest_v, v}}      /* Extend S and T                     */
    for j = 1, ..., n do
        if j ∉ S and c_vj < C_j then
            C_j ← c_vj; closest_j ← v   /* Update C_j and closest_j, ∀j ∉ S          */
```

Complexity order of Prim's algorithm

for $j = 2, \ldots, n$ **do**
  $O(1)$
for $k = 1, \ldots, n-1$ **do**
  for $j = 2, \ldots, n$ **do**
    $O(1)$
  for $j = 2, \ldots, n$ **do**
    $O(1)$

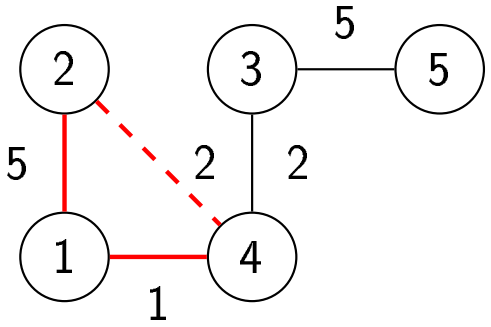
Overall complexity: $O(n^2)$

For sparse graphs, where $m \ll n(n-1)/2$, a more sophisticated data structure leads to an $O(m \log n)$ complexity.

2.2.2 Optimality condition

Definition: Given a spanning tree T, an edge $e \notin T$ is cost decreasing if when e is added to T it creates a cycle C with $C \subseteq T \cup \{e\}$ and $\exists f \in C \backslash \{e\}$ such that $c_e < c_f$.

Example



$c_{24} = 2 < 5 = c_{12}$

Because $c(T \cup \{e\} \backslash \{f\}) = c(T) + c_e - c_f$, if e is cost decreasing, then

$$c(T \cup \{e\} \backslash \{f\}) < c(T).$$

Theorem 2 (Tree optimality condition)

A tree T is of minimum total cost if and only if no cost-decreasing edge exists.

Proof's idea

$\Rightarrow$ If a cost-decreasing edge exists, then T is not of minimum total cost.

$\Leftarrow$ If no cost-decreasing edge exists, then T is of minimum total cost.

Let T^* be a minimum cost spanning tree found by Prim's algorithm.

It can be verified that, by exchanging one edge at a time, T^* can be iteratively transformed into T without modifying the total cost.

Thus, T is also optimal.

Optimality test

The optimality condition allows to verify whether a spanning tree T is optimal: just check that each $e \in E \setminus T$ is not a cost-decreasing edge.

2.2.3 An indirect application: optimal message passing

Given a communication network $G = (N, E)$, we want to broadcast a secret message to all the nodes so that it is not intercepted along any edge.

Problem
Let $p_{ij} \in [0, 1]$ be the probability the message is intercepted along $\{i, j\} \in E$. How to broadcast the message to all the nodes of G, so as to minimize the probability of interception along any edge?

Equivalent to maximize the probability of non-interception along all the edges:

$$\max_{T \in X} \Pi_{\{i,j\} \in T}(1 - p_{ij}).$$

Optimal solution is a spanning tree:

- Broadcasting to all nodes $\Rightarrow$ connected subgraph
- To avoid redundancy and a higher probability of interception $\Rightarrow$ acyclic subgraph.

By applying a monotone increasing function (like log), the objective function values change, but the optimal solutions remain unchanged.

The problem becomes:

$$\max \log \left(\Pi_{\{i,j\}\in T}(1-p_{ij})\right) = \max \sum_{\{i,j\}\in T} \log(1-p_{ij}).$$

which can be solved using a straightforward adaptation of any minimum cost spanning tree algorithm.